mirror-pool

Synchronized behavioral mixing for Solana

Ilan F. (`ilanfds`)
`ilanfdsantos@gmail.com`

July 26, 2026

Abstract

Public ledgers make behavior legible. Every stake, swap, and transfer is permanent, and the graph they form is increasingly read by automated analytics that cluster wallets, attribute identities, and shadow intent in real time. Most privacy tooling answers this by hiding *funds*. We answer a different question: how do you hide *behavior* when the behavior itself has to happen in the open?

mirror-pool is a coordination protocol in which many independent wallets perform the same standardized action inside the same short time window. An observer sees that the action occurred, and can even see every wallet that performed it, but cannot tell which participant genuinely wanted it, nor which one caused the crowd to assemble around it. The construction borrows the skeleton of Tornado Cash [1]—commitments in a Merkle set, per-use nullifiers, and a relayer that pays the fee—but retargets it from "which deposit is being withdrawn" to "who originated a behavioral pattern." It is non-custodial: the protocol never touches a participant's operating capital, and every action runs on the participant's own wallet against the real protocol it interacts with.

We give the threat model, the two independent anonymity properties the design composes, the zero-knowledge statement that hides the initiator, and the on-chain verification path on Solana. We then measure the thing that actually matters. A timing deanonymizer that identifies the initiator of a copy-trading pattern with perfect accuracy drops to random guessing against mirror-pool; a second, cross-round adversary that extracts a persistent power-user under copy-trading does the same. The prototype is written entirely in Rust, with the proof verified on-chain through Solana's `alt_bn128` syscalls.

1 Introduction

The usual pitch for on-chain privacy is about money: break the link between where funds came from and where they went. That is a real problem and it has real tools. It is also not the problem most people actually have. A market maker does not need to launder anything; they need competitors to stop reading their inventory off the tape. An algorithmic trader does not want to hide a balance; they want copy-traders to stop shadowing their entries and eroding an edge that took months to find. A large holder unwinding a position is not committing a crime; they are trying to sell without a wall of front-runners forming the moment their wallet moves. In each case the sensitive object is not a sum of tokens. It is a *decision*, and the leak happens because the decision is legible the instant it touches the chain.

This is the gap mirror-pool targets. The goal is behavioral obscurity: making the actions of protocols, agents, and ordinary users harder for automated chain analysis to read, not by concealing that anything happened, but by drowning any single actor's intent in a crowd of identical, simultaneous, deniable actions. The three ingredients are old ideas in a new arrangement—noise that is statistically indistinguishable from the real signal, plausible deniability for every participant, and a crowd large enough and synchronized enough that membership stops being informative.

The design question is subtle because, unlike a fund mixer, the behavior cannot be hidden inside a contract. Staking has to happen at the staking program. A swap has to hit the venue. The action is public by construction. So the protocol does not try to hide the action; it makes many wallets perform the *same* action at the *same* time, and it hides only two things: who summoned the crowd, and, for any given execution, whether that wallet genuinely wanted the action or was there as cover. The first is a cryptographic problem and we solve it with a zero-knowledge membership proof. The second is free once the first is solved, because uniform simultaneous actions are interchangeable.

The rest of this paper is organized around that split. Section 2 places the work against Tornado Cash and the anonymous-signaling literature. Section 3 states the threat model and what we do and do not promise. Section 4 introduces the two anonymities. Sections 5–8 give the construction: the cryptographic primitives, the round protocol, the proof statement, the on-chain verifier, and the incentive layer. Section 9 analyzes what leaks and what does not. Section 10 measures the protocol against concrete timing adversaries. Section 12 is honest about what is not yet done.

2 Related work

The direct ancestor is Tornado Cash [1]. There, a user deposits a fixed-denomination note whose commitment $\mathsf{H}(k, r)$ enters a Merkle tree; later, anyone can withdraw by proving in zero knowledge that they know the pre-image of *some* leaf in the tree, revealing only a nullifier $\mathsf{H}(k)$ to prevent double-spending. The withdrawal is unlinkable to the deposit, and a relayer submits it so the recipient's gas does not betray them. We reuse this machinery almost verbatim—Poseidon commitments, an incremental Merkle accumulator, a revealed nullifier, a relayer—but the secret it protects is different. Tornado hides *which deposit* is being spent. mirror-pool hides *who originated* a coordinated action. The note is no longer a coin; it is a long-lived membership credential, and the nullifier is derived per round so a member can act once per round without their actions being linkable across rounds.

The idea of proving membership in a set without revealing the member also runs through Semaphore and the anonymous-signaling constructions built on it, and further back through the note-and-nullifier design of Zcash Sapling [5], which is where the pattern of a commitment tree paired with a spend nullifier was worked out in full. The deeper lineage is mixnets: Chaum's untraceable mail [6] is the original statement that you gain anonymity by being one of many indistinguishable messages passing through the same stage at the same time. mirror-pool is, in a sense, a mixnet for behavior rather than for messages, where the "mix" is a synchronized round and the messages are protocol interactions.

What is not borrowed—what we claim as the contribution—is the synchronized round structure itself, the separation of initiator anonymity from intent anonymity, and the adaptation of the Tornado statement to bind a behavioral action rather than a recipient address. The cryptographic primitives underneath (Poseidon [3], Groth16 [2], the BN254 pairing curve [7], Merkle trees [4]) are standard and are used as published.

3 System model and goals

Setting. A set of participants runs software agents. There is a single on-chain program on Solana holding shared state, off-chain services (relayers and keepers) that any party may run, and the real protocols the participants actually interact with (native staking, liquidity venues, and so on). The program never holds a participant's operating capital.

Adversary. We assume a well-resourced observer who reads the entire chain and mempool in real time, runs clustering and copy-trade models over all of it, and can correlate on timing,

amounts, fee payers, counterparties, and sequences of actions. The adversary may join the pool as many times as it likes and watch from the inside. It may also act at the relayer and keeper layer, for instance by selectively refusing to broadcast. We do not assume it can break the soundness of Groth16 over BN254, invert Poseidon, or forge a signature for a key it does not hold.

What we promise. Two properties, made precise in Section 4. First, *initiator anonymity*: for the action a round assembles around, the adversary cannot determine which participant proposed it beyond the anonymity set of eligible proposers. Second, *intent anonymity*: for any single on-chain execution of the round's action, the adversary cannot tell whether that wallet acted because its owner wanted the action or purely to provide cover. Alongside these, the realized size of the crowd in any round is publicly auditable, so a participant can see how much cover actually materialized before relying on it, exactly as a Tornado user can read the anonymity set before withdrawing.

What we do not promise. We do not hide the funds. After a private unstake the resulting balance still sits in a traceable wallet; laundering that balance is a different tool's job. We do not hide that a wallet is a member of the pool—membership is a public deposit, precisely as in Tornado, and treating it as public is a deliberate choice rather than an oversight. We do not address network-layer metadata such as IP addresses, which is mitigated operationally by running many independent relayers and keepers rather than cryptographically.

4 Overview: two anonymities

It is tempting to model this as copy-trading with the roles reversed—a leader acts, followers imitate—and that model is exactly wrong. A visible leader is the most exposed wallet on the chain, and a fixed set of followers is a perfect cluster handed to the analyst for free. The protocol instead composes two anonymity properties that are independent and are protected by different means.

The first is initiator anonymity. Someone has to decide what the crowd does this round. That decision is injected as an *anonymous proposal*: the proposer produces a zero-knowledge proof that they are a member of the pool and reveals a per-round nullifier, but nothing that ties the proposal to a specific wallet. The proposal is submitted through a relayer, so no known wallet even pays for it. This is the part that needs real cryptography, and it is where the Tornado skeleton is reused.

The second is intent anonymity, and it is free. Once the round's action is fixed and many wallets perform it in the same window, any participant who genuinely wanted that action is indistinguishable from the ones providing cover. There is nothing to prove and nothing to hide with a circuit; the anonymity is a consequence of uniformity and synchronization. A whale who wants to unstake does so inside a round where hundreds of others unstake the same standardized amount, and the signal "the whale is exiting" simply ceases to exist.

Separating the two matters because it tells you where to spend effort and where not to. Deciding *what* the crowd rallies around is a governance question, and gating it (say, to a curated set of proposers) is an option a deployment can take without touching the privacy mechanism. Hiding *who* genuinely wanted the action is not a governance question at all; it falls out of the round structure. Conflating the two is what produces designs that are either centralized or insecure.

5 Cryptographic preliminaries

Hash. We use Poseidon [3] over the BN254 scalar field, with the parameters of the circomlib construction [8]. Poseidon is chosen for one reason: it is cheap to prove. A SNARK circuit spends most of its constraints inside hash functions, and a Merkle path of height h evaluates h of them, so a hash designed for arithmetic circuits rather than for CPUs is what makes the whole thing tractable. Writing H for the two-input Poseidon, a membership commitment is

$$C = \mathsf{H}(k, r),$$

where k is a per-member secret and r is randomness. The same H is used for the interior nodes of the Merkle tree and for the nullifier below. It is worth stressing that the identical hash has to be computed in three places—by the agent off-chain, inside the proof circuit, and by the on-chain verifier when it recomputes state—and that any disagreement between them silently breaks every proof. We return to this in Section 11.

Membership accumulator. Members are accumulated in a fixed-height incremental Merkle tree with Poseidon nodes, in the style of Tornado's `MerkleTreeWithHistory`. Empty leaves are the field zero, and the tree keeps a rolling window of recent roots so that a proposal may reference a slightly stale root without being invalidated by concurrent deposits. Inserting a leaf costs h hashes and updates the root in place.

Nullifiers. A nullifier is a value revealed when acting that lets the verifier detect a repeat without learning who acted. Tornado derives it from k alone, because a note is spent once. Our members act many times, so we bind the nullifier to the round:

$$nf = \mathsf{H}(k, rid).$$

Deriving it from k (and not from any proposer credential) means distinct members produce distinct nullifiers, so at most one proposal per member per round gets through; deriving it with rid means the nullifiers of one member across different rounds are uncorrelated, which is what allows long-lived membership with per-round unlinkability.

Proof system. We use Groth16 [2] over BN254. Groth16 gives constant-size proofs and, more importantly here, a verification equation that Solana can evaluate: the runtime exposes the BN254 pairing and group operations as syscalls, and a Groth16 check is a fixed handful of them. The cost is a per-circuit trusted setup, which we discuss in Section 12.

6 The protocol

6.1 Joining

A member samples (k, r), computes $C = \mathsf{H}(k, r)$, and submits C to the program, which appends it to the Merkle tree. This is the only interaction that looks like a Tornado deposit, and like a Tornado deposit it is public: anyone can see that a wallet joined. What no one can see is any later link between this wallet and the proposals it makes. Membership is long-lived; a single join lets the member participate in arbitrarily many future rounds. The note (k, r) is the member's secret, and losing it costs only the ability to propose—never funds, which the protocol never holds.

6.2 Rounds

Time is divided into rounds, and a round moves through a small state machine driven by slot deadlines:

$$\mathsf{Propose} \to \mathsf{Seal} \to \mathsf{Commit} \to \mathsf{Threshold} \to \mathsf{Execute} \to \mathsf{Close}.$$

During Propose, anonymous proposals arrive and one action is settled on. Seal freezes that action; nothing about it can change afterward, which matters because participants are about to sign transactions that hard-code it. During Commit, participants sign up to execute. Threshold is the decision point: if the number of committed participants reaches the round's target N, the round goes to Execute; otherwise it aborts and nobody acts. Execute is the synchronized window in which the crowd performs the action. Close finalizes the round's bookkeeping.

The abort path is not an afterthought; it is the mechanism that keeps a participant from ever being exposed in a thin crowd. Because the threshold is evaluated collectively, either enough wallets showed up and they all act together, or too few did and *none* of them act. A participant who signed up optimistically is never left executing a conspicuous action alone. This recreates, inside a single round, the property that makes Tornado usable: the anonymity set is visible before you commit to relying on it. The commit phase is that visible set; the execute phase is the withdrawal that only fires once the set is large enough.

6.3 Anonymous proposal

To make the crowd rally around an action a, a member produces a Groth16 proof of the following statement. Public inputs are the membership root R, the nullifier nf, the round identifier rid, and the action a; the witness is the note and a Merkle opening.

$$\begin{aligned} S_{\text{propose}}[R, nf, rid, a] = \{\ &\text{I know } k, r, \ell, \{o_i\} \text{ such that} \\ &C = \mathsf{H}(k, r), \\ &\text{the opening } \{o_i\} \text{ carries } C \text{ to } R \text{ at leaf } \ell, \\ &nf = \mathsf{H}(k, rid)\ \}. \end{aligned}$$

This is the Tornado withdrawal statement with two changes. The recipient and fee are gone, and in their place the action a is bound as a public input. Binding a makes the proof non-malleable with respect to it: a relayer handling the proof cannot swap a for a different action, because the proof was produced for this a and the verifier checks it against the public inputs it is given. The circuit itself is small—a commitment hash, a Merkle path of Poseidon hashes with a conditional swap at each level selected by the leaf-index bits, and a nullifier hash—and it is exactly the canonical membership circuit plus the action binding.

By default any member may run this. One proposal per member per round is enforced by the nullifier, and that is the only gate; there is deliberately no notion of a privileged proposer in the base protocol. This maximizes the anonymity set of the initiator and keeps the circuit minimal. A deployment that wants to restrict who may steer the crowd can layer a credential on top—the design accommodates a shared or per-proposer key, or a bonded proposer set—but none of that is required for safety, and none of it changes the privacy of the participants who merely provide cover.

The program verifies the proof, checks that nf has not already been used this round (a per-round marker prevents replay), confirms that R is one of the recently recorded roots (so a prover cannot substitute a tree of their own making), and seals a as the round's action. The proposal is delivered through a relayer, so the proposer's own wallet is never the fee payer on-chain and there is no wallet-level footprint of the act of proposing.

6.4 Commitment and synchronized execution

Behavior is not fungible the way a fixed-denomination note is. To "unstake 100 units," each participant must act on their own staked position, from their own wallet, at the real staking program. The protocol coordinates timing and uniformity; it never routes funds. This has an immediate consequence that is easy to get wrong: the keeper that fires the round must *not* be the one interacting with the protocols. If a single keeper performed every action, the chain would show one wallet doing everything, which is no cover at all and would require custody of everyone's funds. The crowd exists precisely because many distinct wallets each act on their own behalf. The keeper only decides *when*.

The mechanism that lets it decide when, without the keeper being able to alter or steal anything, is Solana's durable nonce. A participant signs their execution transaction *once*, at commit time, against a nonce account they own. Solana transactions normally expire in about a minute because they carry a recent blockhash; a durable-nonce transaction instead carries the nonce account's stored value and remains valid until that nonce advances. The participant hands the signed transaction to a keeper, and at execute time the keeper broadcasts it. The keeper holds a transaction the participant already signed: it cannot change the action (the signature would break), it cannot move funds anywhere except where the scoped action sends them, and the worst it can do is decline to broadcast, which merely thins that participant's round. And because only the participant—the nonce authority—can advance the nonce, advancing it is a clean, unilateral opt-out: do so and the pre-signed transaction is dead.

Committing is a per-wallet, openly signed action. A participant registers their commit once per round, a marker prevents a single wallet from inflating the count, and the commit counter that the threshold reads therefore reflects distinct participants. We considered making the commit itself anonymous and decided against it for the base protocol, for the same reason Tornado leaves deposits public: providing cover is a public act by design, the anonymity that matters attaches to the *initiator*, and a signed commit reveals nothing about them. There is no on-chain link from the anonymous proposal to any committing wallet, so a member who proposes and then commits as one of the crowd is not exposed by committing. Hiding crowd membership as well is a strictly stronger goal, and we treat it as future work in Section 12.

7 The action vocabulary, and where it is enforced

Open strangers are being asked to mirror an action chosen by a proposer they do not know and cannot see. What stops that action from being a trap—a swap into a worthless token, an approval to a drainer? The answer is not to gate who proposes; a malicious insider is, by definition, an authorized proposer. The answer is to constrain *what* can be proposed. A pool fixes a vocabulary of whitelisted, non-custodial actions at standardized denominations—stake and unstake in round-number amounts, and nothing that grants an arbitrary approval or performs an arbitrary call. Within such a vocabulary a harmful action is simply not expressible, and standardized amounts also serve anonymity directly, since non-standard sizes would fingerprint a participant the way a non-standard note would in Tornado.

We are precise about where this constraint currently lives, because it is a place where a careful reader should push. In the present design the on-chain action is an opaque field: the program binds and seals whatever action a valid proof carries, and does not itself check that action against a vocabulary. The enforcement is applied off-chain, by the participating agents. An agent only mirrors an action that lies inside a policy its operator has accepted, so an out-of-vocabulary action finds no takers, fails to reach the threshold, and aborts the round at no cost to anyone. This is a sound design—the poison finds no crowd—but it relies on honest agent software rather than on the program. Lifting the vocabulary check on-chain, so the program rejects an out-of-vocabulary action regardless of the agents, is a planned hardening and not a

change to the protocol's shape.

8 Incentives

A crowd is only private if enough wallets show up, and showing up costs transaction fees and, for actions that are not net-neutral, some slippage. Left alone this is a public-goods problem: everyone benefits from a large crowd and no one is individually paid to be in it. Three ingredients address it, and they compose.

The first is a market. The party that needs cover—the one whose real intent is being hidden—pays a fee, and that fee is distributed to the wallets that provided cover. This is Tornado's relayer fee generalized from one relayer to a crowd: demand pays supply, and providing cover turns from a cost into a small profit. The second is non-monetary and is what the prototype implements today: a cover-credit ledger. Executing a round earns a credit; a member who wants the crowd to cover their own action spends credits to summon it. "Provide cover to get cover" keeps the economy closed and, because credits accrue to wallets that are openly acting as cover anyway, it avoids creating a fresh cluster of reward wallets to track. The third is not a mechanism but a design bias: choosing a vocabulary of actions people would plausibly perform regardless—staking for yield, providing liquidity for fees—so that the marginal cost of being cover is close to a transaction fee, which keeps the required subsidy or credit balance small.

There is a hard problem lurking here and we do not pretend to have solved it. Rewarding a *specific* participant for executing, in a way that cannot be linked back to their wallet, is in tension with anonymity: the moment you pay a wallet for cover you have labeled it. The clean version needs an unlinkable proof of participation, which is genuine research and which we flag as such. The credit ledger sidesteps this by treating cover providers as the public actors they already are, and by measuring the property that actually protects the initiator—the size of the crowd—as a public count that requires no per-member attestation at all.

9 Security and anonymity analysis

The initiator is hidden among eligible proposers by the membership proof, and the relayer removes the fee-payer footprint that would otherwise mark a wallet as having proposed. Off-chain, an observer cannot enumerate who holds a proposer credential (when one is used at all), so the effective proposer set is not trivially small. The participant with genuine intent is hidden among everyone who executed the round's action, and the size of that set is a public count of identical on-chain actions in the window—which means the metric that protects everyone is an aggregate, not an identification, and measuring it never itself creates a link.

The leaks worth naming are the ones a real analyst reaches for. Amount is handled by fixed denominations. Timing is handled by executing inside a tight window with jitter, so the ordering of executions within the window carries no information about who initiated. The fee payer of a proposal is handled by the relayer. A thin crowd is handled by the collective threshold and the abort path, so no one is ever alone. Cross-round correlation is bounded by nullifiers that are uncorrelated across rounds and by the option to rotate execution wallets. The one residual surface is the keeper: it learns, before broadcasting, that a given wallet intends to perform the round's action. This is the same trust surface as a Tornado relayer, which sees a withdrawal before submitting it, and it is mitigated the same way, by running many keepers and by letting a participant broadcast their own transaction as a fallback.

We are candid about the boundary the previous section drew. Because commits are openly signed, an adversary can see the composition of a round's crowd, and a crowd that always contains the same wallets could be clustered as a set of regular participants. That is a statement about *membership*, which we have declared out of scope, and not about *initiation*, which is what the adversary actually wants and does not get. It is exactly the situation in Tornado, where the

set of depositors is public and the value of the system is that you cannot say which depositor withdrew.

10 Evaluation

The claim that survives or sinks the whole design is that the crowd defeats chain analysis rather than merely looking like it should. We test it directly, by simulating rounds under two behaviors and running the same deanonymizer against both. Under *copy-trading*, the initiator acts first and the followers react, so the initiator is the earliest executor. Under *mirror-pool*, every participant, the initiator included, executes at an independent uniform time inside the window. The adversary is the obvious one: name the earliest executor as the initiator.

With a crowd of $N = 50$ over 5000 rounds, random guessing scores $1/N = 0.0200$. The results are in Table 1. The identical heuristic that identifies the initiator every single time under copy-trading collapses to the random baseline under mirror-pool. The ordering signal that leaks the initiator is not obscured; it is erased, because under uniform simultaneous execution the earliest wallet carries no information at all.

Adversary	Copy-trading	mirror-pool
Earliest-executor (per round)	1.0000	0.0198
Most-frequently-earliest (cross-round, power initiator)	0.5116	0.0138

Table 1: Initiator-attribution accuracy. Random guessing is $1/N = 0.02$. Both a per-round and a cross-round timing adversary succeed against copy-trading and collapse to random against mirror-pool.

One adversary that fails could be a straw man, so we add a second and stronger one. Suppose a persistent "power initiator" starts half the rounds—the kind of repeat pattern a longitudinal analyst hunts for. A cross-round adversary that tracks which wallet is most often the earliest executor, and blames it for every round, extracts that power initiator with better than 0.5 accuracy under copy-trading. Under mirror-pool the same attack falls to 0.0138: because earliness is uniform, the power initiator is early no more often than anyone else, and the longitudinal signal it depends on is gone. Both experiments are seeded and reproducible, and both properties are asserted by tests that fail if attribution ever climbs back toward the copy-trading numbers.

The evaluation is deliberately narrow. It isolates timing and execution-ordering attribution, which is the signal that leader/follower schemes leak and the one the synchronized round is built to kill. A real adversary combines signals, and the other signals are handled out of band—amount by fixed denominations, the proposal footprint by the relayer, thin crowds by the threshold. What the experiment establishes is precise and checkable: mirror-pool removes the ordering signal that the most direct timing adversary relies on, and it does so by construction, measured here rather than asserted.

11 Implementation notes

Although the focus of this document is the construction rather than the code, two implementation facts are load-bearing for the security argument and deserve a sentence each. First, the same Poseidon has to hold across three domains, and we do not merely hope that it does: the off-chain hash, the on-chain hash computed by Solana's syscall, and the in-circuit hash are cross-checked against one another, and the whole stack is pinned to an external reference by a known-answer test against the canonical circomlib value of $\mathsf{H}(1, 2)$. If any of the three diverged, membership proofs would not verify, and the tests would catch it before anyone

shipped. Second, the proof that an agent produces off-chain is verified by exactly the same Groth16 verifier that runs inside the program, so the byte-level encoding of the proof and verifying key—the endianness, the negation of the first proof element, the ordering of the pairing's second-group coordinates—is validated against the on-chain verifier's own implementation before it ever reaches the chain. The prototype is written entirely in Rust: the circuit and prover on arkworks [9], the on-chain verification through Light Protocol's `groth16-solana` [10] over Solana's `alt_bn128` syscalls, and the Poseidon parameters from the same source as circomlib so the constants are not transcribed by hand.

12 Limitations and future work

The most important limitation is the trusted setup. Groth16 requires a per-circuit setup, and a single party who runs it and keeps the toxic waste can forge proofs. The prototype ships a development key from a one-party setup, which is fine for testing and unacceptable for production; a real deployment needs a multi-party ceremony of the kind used for zk-SNARK parameters [11], and until that exists the system should be treated as experimental. Three other items are tracked as work rather than as claims. The action vocabulary is enforced by agents and not yet by the program (Section 7). The monetary cover market is designed but the prototype implements only the credit ledger (Section 8). And the keeper and relayer roles want to be decentralized, so that no single party can censor a round.

Beyond hardening, the interesting open problem is the one flagged in Section 8: an unlinkable proof of participation, which would let the protocol reward or penalize specific participants without labeling their wallets, and which would also be the ingredient needed to hide crowd membership rather than merely initiation. That is where the design stops being an engineering exercise and becomes research.

13 Conclusion

mirror-pool takes a construction built to hide money and turns it to hide behavior. The move that makes it work is the recognition that you do not need to hide the action at all—only the identity of the one who wanted it—and that once a crowd performs the same action at the same time, that identity dissolves into the crowd for free. The cryptography is deliberately conventional; the contribution is the arrangement. What we can show, and did, is that the arrangement does the one thing it exists to do: a timing adversary that reads the initiator straight off a copy-trading tape is reduced to guessing.

References

[1] A. Pertsev, R. Semenov, and R. Storm. *Tornado Cash Privacy Solution, Version 1.4*. 2019.

[2] J. Groth. On the Size of Pairing-Based Non-interactive Arguments. *EUROCRYPT 2016*, LNCS 9666, pp. 305–326. Springer, 2016.

[3] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. *30th USENIX Security Symposium*, 2021.

[4] R. C. Merkle. A Digital Signature Based on a Conventional Encryption Function. *CRYPTO 1987*, LNCS 293, pp. 369–378. Springer, 1988.

[5] D. Hopwood, S. Bowe, T. Hornby, and N. Wilcox. *Zcash Protocol Specification*. Electric Coin Company.

[6] D. Chaum. Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms. *Communications of the ACM*, 24(2):84–90, 1981.

[7] P. S. L. M. Barreto and M. Naehrig. Pairing-Friendly Elliptic Curves of Prime Order. *SAC 2005*, LNCS 3897, pp. 319–331. Springer, 2006.

[8] iden3. *circom and circomlib*. `https://github.com/iden3/circomlib`.

[9] arkworks contributors. *arkworks: A Rust ecosystem for zkSNARK programming*. `https://arkworks.rs`.

[10] Light Protocol. *groth16-solana and light-poseidon*. `https://github.com/Lightprotocol`.

[11] S. Bowe, A. Gabizon, and I. Miers. Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model. *IACR ePrint* 2017/1050.